Общие требования к выполнению лабораторных работ

1. Язык и платформа

1. Лабораторные работы выполняются **на языке Java**.
2. Рекомендуемая версия Java: Java **17** или Java **21**.
3. Использование сторонних библиотек разрешено для решения связанных задач, а не для создания приложения на основе одной библиотеки (например, командного интерпретатора).

2. Система сборки

1. Проект **обязательно** должен использовать систему сборки:
2. **Gradle** или
3. **Maven**.
4. Сборка проекта должна выполняться командой: gradle build или mvn package
5. Проект **не должен требовать IDE для сборки**.
6. Все зависимости должны подтягиваться автоматически.

3. Контроль версий (Git)

1. Исходный код проекта **обязательно** должен храниться в системе контроля версий **Git**.
2. Репозиторий должен содержать: историю коммитов, читаемые сообщения коммитов.
3. Запрещается: сдавать проект одним коммитом, загружать архивы с кодом вместо репозитория.
4. Минимум требований к коммитам: не реже одного осмысленного коммита на логически завершённый этап работы, сообщения коммитов должны отражать суть изменений (не fix, upd, asdf).

4. Структура проекта

1. Проект должен иметь **понятную пакетную структуру**, например: domain, service, repository, cli / ui
2. Классы предметной области должны быть **отделены от UI, ввода/вывода и работы с БД**.

3. Запрещается размещать всю логику в одном классе или пакете.

5. Нейминг и читаемость кода

1. Имена классов, методов и полей должны: отражать смысл, быть на английском языке, использовать camelCase / PascalCase.
2. Запрещаются имена вида: a, tmp, data1, qwerty, Test, MyClass, Class1.
3. Названия должны быть **предметно-ориентированными**: Sample, а не Obj; addMeasurement, а не doStuff

6. Работа с ошибками

1. Все ошибки ввода и выполнения должны: обрабатываться, сопровождаться **понятными сообщениями**.
2. Сообщения об ошибках должны объяснять: **что произошло**, **что пользователь сделал не так**.
3. Вывод stacktrace пользователю запрещён (допускается только во время разработки).

7. Валидация данных

1. Все данные, вводимые пользователем, должны проверяться.
2. Валидация должна выполняться: до сохранения данных, до выполнения операций над ними.
3. Запрещается полагаться на “корректный ввод” от пользователя.

8. Повторное использование и дублирование

1. Повторяющийся код должен выноситься: в методы, в утилитарные классы.
2. Копипаст без объяснения **снижает оценку за понимание**.

9. Конфигурация и данные

1. Пути к файлам, параметры подключения к БД и прочие настройки: не должны быть захардкожены в коде.
2. Конфигурация должна быть вынесена: в файл, или в переменные окружения.

10. Запуск и проверяемость

1. Проект должен:
2. корректно собираться, запускаться по инструкции.

3. В репозитории должен быть файл README.md, содержащий: краткое описание проекта, требования к запуску, команды сборки и запуска.

11. Самостоятельность выполнения

1. Студент должен понимать и уметь объяснить **любой фрагмент своего кода**.
2. Неспособность объяснить код: резко снижает оценку, может привести к аннулированию результата этапа.
3. Использование внешних источников допускается, но: код должен быть адаптирован, студент обязан понимать его работу.

```
}
package ru.yourteam.lab.domain;

public enum MaintenanceType {
    SERVICE,
    REPAIR
}
```

Предметная область 4: Партии экспериментов и результаты серий

Система хранит:

- **Experiment** — эксперимент (название/цель)
- **Run** — запуск/прогон (одна попытка)
- **RunResult** — результаты (числа, заметки, файл-ссылка)

Команды (10)

1) exp_create

Пример:

```
> exp_create
Название: Nitrate removal test
Описание (можно пусто): batch adsorption series
OK experiment_id=2
```

Ошибки: пустое имя

2) exp_list [--mine]

```
> exp_list
ID  Name
2   Nitrate removal test
```

3) exp_show <experiment_id>

```
> exp_show 2
Experiment #2
name: Nitrate removal test
runs: 3
```

4) exp_update <experiment_id> field=value ...

Поля: name, description

```
> exp_update 2 name="Nitrate removal test (v2)"
OK
```

5) run_add <experiment_id>

```
> run_add 2
Run name: Run-2026-02-03-A
Operator: yarus
OK run_id=11
```

Ошибки: experiment не найден

6) run_list <experiment_id> [--last N]

```
> run_list 2 --last 2
ID  Run name            Operator  Time
11  Run-2026-02-03-A     yarus     2026-02-03 18:30
10  Run-2026-02-02-B     yarus     2026-02-02 14:10
```

7) run_show <run_id>

```
> run_show 11
Run #11
experiment_id: 2
name: Run-2026-02-03-A
results: 4
```

8) res_add <run_id>

```
> res_add 11
Параметр: NITRATE
Значение: 12.4
Единицы: mg/L
Комментарий (можно пусто): after 60 min
OK result_id=88
```

Ошибки: значение не число, пустые unit

9) res_list <run_id> [--param PARAM]

```
> res_list 11
ID  Param    Value  Unit  Comment
88  NITRATE  12.4   mg/L  after 60 min
```

10) exp_summary <experiment_id>

Что делает: простая сводка: по каждому параметру min/max/avg по всем runs.

```
> exp_summary 2
NITRATE: count=10 min=8.1 max=15.0 avg=11.2
PH:      count=10 min=6.8 max=7.3  avg=7.0
```

Каркас классов

```
package ru.yourteam.lab.domain;

import java.time.Instant;

public final class Experiment {
    // Уникальный номер эксперимента. Программа назначает
сама.
    public long id;

    // Название эксперимента. Нельзя пустое. До 128 символов.
    public String name;

    // Описание (кратко “что делаем”). Можно пусто. До 512
символов.
    public String description;

    // Кто создал (логин). На ранних этапах можно "SYSTEM".
    public String ownerUsername;

    // Когда создан. Программа ставит автоматически.
    public Instant createdAt;

    // Когда изменяли. Программа обновляет автоматически.
    public Instant updatedAt;
}
package ru.yourteam.lab.domain;

import java.time.Instant;

public final class Run {
    // Уникальный номер запуска. Программа назначает сама.
    public long id;

    // К какому эксперименту относится (id эксперимента).
    // Должен ссылаться на реально существующий Experiment.
    public long experimentId;

    // Название запуска reminder: “Run-2026-02-03-A”. Нельзя
пустое. До 128 символов.
    public String name;
```

```
    // Кто выполнял запуск (логин или имя). Нельзя пустое. До
64 символов.
    public String operatorName;

    // Когда запуск зарегистрирован. Программа ставит
автоматически.
    public Instant createdAt;
}
package ru.yourteam.lab.domain;

import java.time.Instant;
public final class RunResult {
    // Уникальный номер результата. Программа назначает сама.
    public long id;

    // К какому запуску относится (id запуска).
    // Должен ссылаться на реально существующий Run.
    public long runId;

    // Что измеряли (PH/CONDUCTIVITY/NITRATE...). Выбирается
из списка MeasurementParam.
    public MeasurementParam param;

    // Числовое значение результата.
    public double value;

    // Единицы (например "mg/L"). Нельзя пустое. До 16
символов.
    public String unit;

    // Комментарий (например “after 60 min”). Можно пусто. До
128 символов.
    public String comment;

    // Когда добавили результат. Программа ставит
автоматически.
    public Instant createdAt;
}
```

Варианты для ЛР2

Варианты сохранения коллекции в файл:

№	Вариант
1	JSON
2	XML
3	CSV
4	Сериализация

Варианты пользовательского интерфейса (JavaFX):

№	Вариант
1	Таблица
2	Master-detail
3	Карточки
4	Вкладки

1. Таблица

Основной экран представляет собой таблицу (TableView) со списком объектов.
Пользователь может добавлять, изменять и удалять объекты через формы или диалоги.
Обновление данных выполняется вручную по кнопке **Refresh**.

2. Master–Detail

Интерфейс разделён на две части:
слева — список или таблица объектов, справа — подробная информация о выбранном объекте.
При смене выбора обновляется правая часть.
Кнопка **Refresh** перечитывает данные и обновляет список объектов.

3. Карточки

Объекты отображаются в виде списка карточек, каждая карточка содержит основные поля объекта.
При выборе карточки можно открыть подробную информацию или форму редактирования.

Обновление списка карточек выполняется вручную.

4. Вкладки

Интерфейс разделён на вкладки (TabPane), каждая вкладка содержит отдельный экран (таблицу, список или форму).
Обновление данных выполняется вручную отдельно для каждой вкладки.

Задание ЛР 1

Этап 1: Доменные классы + валидаторы + операции над коллекцией

1) Доменные классы предметной области

1. Реализовать **все классы сущностей** предметной области.
2. В каждом классе должны быть:
   - **поля** (по предметной области),
   - **конструктор**, который создаёт объект из набора значений,
   - **геттеры/сеттеры**,
3. Поле id:
   - назначается **программой**, а не вводится пользователем,
   - уникально в пределах коллекции.

Важно: не просто “описать поля”, а **сделать рабочие классы**, которые реально используются в коде команд. *Обратите внимание на модификаторы полей классов – они должны быть private.*

2) Валидаторы

1. Для каждой сущности сделать **валидатор** (отдельный класс или набор методов или проверка внутри setter и конструкторов классов), который проверяет ограничения полей.
2. Валидация должна быть вызвана:
   - при **создании** объекта (до добавления в коллекцию),
   - при **изменении** объекта (до сохранения изменений).
3. При ошибке валидации:
   - операция отменяется,
   - пользователю выводится **понятная причина**, например:
     - Ошибка: name не может быть пустым
     - Ошибка: значение должно быть больше 0

3) Коллекция объектов и операции предметной области

1. Данные хранятся **только в памяти**.
2. Реализовать “менеджер/сервис” коллекции (например SampleService, Storage, CollectionManager), который содержит:

    - структуру хранения (List/Map),
    - генерацию id,
    - методы работы с коллекцией.
3. Обязательные методы менеджера:
    - add(...) — создать и добавить объект (с валидатором)
    - getById(id)
    - list() / getAll()
    - update(id, ...) (с валидатором)
    - remove(id)
4. Дополнительно реализовать методы менеджера, которые необходимы для реализации специфических команд из вашей предметной области

То есть: команды должны будут вызывать **методы менеджера**, а не напрямую работать с коллекцией из CLI. *Обратите внимание – скорее всего вам потребуется не одна коллекция, а на каждую сущность по коллекции.*

Этап 2: Выполнение команд

1) Командный интерпретатор (CLI)

1. Программа читает и выполняет команды из консоли в цикле до exit.
2. Обязательные общие команды:
    - help (список команд)
    - exit
3. Реализовать команды из предметной области
4. Хотя бы одна команда должна быть **интерактивной** (с доп. вводом по полям, см. предметную область), то есть:

    - пользователь вводит команду,
    - программа задаёт вопросы по полям,
    - пользователь вводит значения по одному.

    Обычно такой командой является add/create из вашей предметной области

2) Обработка ошибок

Программа не должна падать. Например, должны быть обработаны ошибки:

- неизвестная команда
- неправильное число аргументов
- id введён не числом
- объект с таким id не найден
- ошибки валидации полей

Более подробно см. в предметной области. Ошибки должны выводиться понятным сообщением.

3) Что должно быть видно в коде

Проект должен содержать **минимум** подобные части:

- domain — классы сущностей
- validation — валидаторы
- service (или manager) — операции над коллекцией + генерация id + вызовы валидаторов
- cli — разбор команд и вызов методов сервиса

Задание ЛР 2

Этап 3: Файл (сохранение/загрузка)

1) Что нужно реализовать

1. Реализовать сохранение текущих данных из памяти в файл командой:

- save <path> (или фиксированный путь, но лучше через параметр)

2. Реализовать загрузку данных из файла командой:

- load <path>

3. После загрузки данные в программе должны быть заменены на загруженные.
4. Должна быть возможность указать путь к файлу при запуске программы в аргументах командной строки

На этом этапе данные по-прежнему хранятся в памяти, файл — только для сохранения/восстановления.

2) Где это должно быть реализовано

- В отдельном слое/модуле хранения, например:
  - storage / persistence / file
- Должны быть классы:
  - FileStorage (или FileRepository) — читает/пишет файл
  - FileValidator — проверяет корректность содержимого файла
- Команды save/load **не должны** напрямую читать/писать файл: они вызывают методы сервиса/хранилища.

3) Требования к формату файла

Формат файла выбирается вариантом (например JSON или CSV).
Главное требование: **формат должен быть однозначно читаемым обратно** и позволять хранить все поля сущностей.

4) Проверка валидности файла

При загрузке программа должна проверить:

1. **Файл читается**

- файл существует / доступен
- формат корректный (не “битый”)

2. **У объектов есть нужные поля**

- нет пропусков обязательных полей
- типы значений корректны (число/строка/список)

3. **Соблюдены ограничения полей**

- те же правила, что и в валидаторах этапа 1
  (не пустые строки, числа в диапазоне, значения из допустимого списка)

4. **Целостность данных между сущностями**

- если есть ссылки (например sampleId), они должны указывать на реально существующий объект
- id не должны повторяться

5) Поведение при ошибках загрузки

Если файл невалиден:

- загрузка **не должна частично менять данные**
- должны сохраниться прежние данные в памяти
- пользователь должен получить понятное сообщение:
  - что именно не так (минимум одна причина)

Пример:

- Ошибка загрузки: measurement.sampleId=12 ссылается на несуществующий sample
- Ошибка загрузки: поле name пустое у объекта id=5

Этап 4: JavaFX UI

1) Что нужно реализовать

1. Реализовать JavaFX приложение (одно главное окно).
2. UI должен показывать данные из коллекции (список объектов).
3. UI должен позволять выполнять операции предметной области.

2) Варианты UI

Вид UI выбирается вариантом

3) Ручное обновление

Список и содержимое объектов в UI **не должен обновляться автоматически** после изменения данных другими пользователями.

Обязательные требования:

1. В UI должна быть кнопка **Refresh**.
2. Только при нажатии **Refresh** UI перечитывает данные из “источника” и перерисовывает список.

Допустимая реализация:

- очистить локальную копию и заполнить ей заново

4) Где должна быть логика UI

- UI (JavaFX) вызывает методы **сервиса/менеджера**, а не работает напрямую с коллекцией.
- Доменная логика и валидация остаются в сервисе/валидаторах.

5) Ошибки в UI

Ошибки должны показываться пользователю:

- через Alert/диалог
- понятным текстом (без stacktrace)

6) Реализация UI

Интерфейс не должен зависать при долгих операциях, допустим показывать progress bar пользователю. Используйте Platform.runLater(…) при необходимости.